

RAPPORT D'ARCHITECTURE LOGICIELLE

Ségrégation Multi-Schémas, Intégrité Référentielle & Modularité
(PostgreSQL)

Projet : IncidentFlow V2
Auteur : Anas Benhammou
Standard : Domain-Driven Data
Isolation

1. Problématique Initiale : Limites du Schéma Monolithique

Lors de la phase initiale de la version V2, l'ensemble des modules d'infrastructure (le gestionnaire d'identités Keycloak et le moteur de workflow Flowable) ainsi que le modèle métier applicatif cohabitaient au sein de l'unique schéma par défaut de PostgreSQL (`public`).

Cette centralisation présentait des faiblesses architecturales manifestes : pollution de l'espace de nommage (plus de 150 tables hétérogènes entremêlées dans les outils d'administration), couplage fort entre composants tiers et applicatifs, et impossibilité de définir des politiques de sauvegarde ou de droits granulaires.

2. Architecture Cible : Partitionnement en 4 Schémas Dédiés

Pour répondre aux normes industrielles d'isolation des domaines, la base de données IncidentFlow_db a été structurée en **quatre compartiments logiques étanches** :

Schéma PostgreSQL	Domaine de Responsabilité	Fournisseur / Moteur	Tables Actives
<code>public</code>	Schéma technique standard PostgreSQL	SGBD Core	0 table (Maintenu 100% vierge)
<code>incidentflow</code>	Entités Métier (Incidents, Commentaires, Pièces jointes)	Hibernate 6 / JPA (Spring Boot)	11 tables
<code>flowable</code>	Orchestration BPMN 2.0, Runtime et Audit Log natif	Flowable Engine 7.0.1	47 tables
<code>keycloak</code>	Identités, Cryptographie, Sessions JWT, Rôles RBAC	Keycloak 21.1.1 (Quarkus)	92 tables

3. Mécanisme de Transition : Configuration Déclarative vs "Réinstallation"

Il est capital de préciser qu'**aucune réinstallation logicielle lourde** n'a été opérée. Dans un écosystème conteneurisé moderne (Docker), la ségrégation repose entièrement sur le paradigme de l'**Infrastructure as Code (IaC) déclarative** :

- Auto-Provisionnement Keycloak** : En spécifiant `KC_DB_SCHEMA=keycloak` dans `docker-compose.yml`, le moteur interne Liquibase de Keycloak a détecté le schéma cible et a automatiquement créé les 92 tables au redémarrage, important de façon transparente le royaume (`realm-export.json`).
- Auto-Provisionnement Flowable & Hibernate** : Par injection des propriétés `flowable.database-schema=flowable` et `spring.jpa.properties.hibernate.default_schema=incidentflow`, Spring Boot a délégué la création des tables à leurs moteurs respectifs.
- Purge finale du schéma public** : Une commande DDL atomique (`DROP SCHEMA public CASCADE; CREATE SCHEMA public;`) a nettoyé l'ancien schéma encombré, le laissant à 0 table.

4. Intégrité Référentielle : Préservation des Clés Étrangères

Une préoccupation majeure lors du déplacement vers des schémas multiples concerne la préservation des contraintes relationnelles. L'audit du dictionnaire de données PostgreSQL (`information_schema.table_constraints`) atteste de la parfaite intégrité du modèle :

Schéma Relationnel	Nombre de Clés Étrangères SQL Actives	Exemples de Relations Préservées
incidentflow	12 contraintes	<code>comments.incident_id → incidents.id</code> , <code>incidents.author_id → users.id</code>
flowable	46 contraintes	<code>act_ru_task.execution_id_ → act_ru_execution.id_</code>
keycloak	74 contraintes	<code>user_role_mapping.user_id → user_entity.id</code>

Le Concept Fondamental de Clé Étrangère Conceptuelle (Soft Foreign Key) :

Entre le schéma métier (*incidentflow*) et le moteur de workflow (*flowable*), la liaison est assurée par la colonne *process_instance_id*. Dans une architecture orientée services, **il est formellement proscrit** de poser une contrainte SQL physique dure (`REFERENCES flowable...`) vers un framework tiers. Une telle contrainte interdirait la purge des historiques, bloquerait les migrations indépendantes et briserait le principe de découplage strict (Bounded Context - DDD).

5. Cycle de Vie BPMN : Résolution de l'Anomalie des Versions Multiples

Lors des premières itérations, l'affichage de "trois modèles" dans la base de données s'expliquait par deux mécanismes natifs de Flowable :

- **Le Versioning Automatique** : Flowable ne supprime jamais un modèle modifié ; il incrémente une colonne de version (`version_ = 1, 2, 3` dans `act_re_procdef`) afin que les instances d'incidents déjà en cours puissent s'achever sans interruption de service.
- **Découplage Modeler / Runtime** : La suppression d'un dessin dans l'interface de modélisation (Flowable Modeler) ne désinstalle pas le binaire exécutable déployé dans le moteur (Engine).

La transition vers le schéma propre *flowable* a permis une réinitialisation intégrale : la table `flowable.act_re_procdef` ne contient désormais plus qu'un **unique processus actif certifié** (`incidentWorkflow:1`), aligné avec le fichier XML source du projet.

6. Avantages Comparatifs pour l'Entreprise

1. Sécurité & Moindre Privilège (RBAC DB)

Chaque composant dispose d'un rôle SQL restreint à son schéma. L'application métier ne peut ni lire ni altérer les mots de passe et credentials de Keycloak.

2. Sauvegardes Sélectives (Backup & Restore)

Possibilité d'exporter uniquement le schéma métier avec `pg_dump -n incidentflow` (quelques kilo-octets) sans répliquer 92 tables de sessions IAM temporaires.

3. Cycle de Vie et Mises à Niveau Découplés

Mise à niveau (Upgrade) de Flowable ou Keycloak sans aucun risque d'impact de schéma ou de régression sur les entités de gestion d'incidents.

4. Conformité & Gouvernance des Données (RGPD)

Séparation physique stricte entre les données identitaires (schéma *keycloak*) et les données opérationnelles techniques (schéma *incidentflow*).

7. Procédure de Renommage et Haute Flexibilité

Si l'organisation souhaite adapter la nomenclature de ses domaines (par exemple, renommer `incidentflow` en `core_incidents` ou `keycloak` en `iam_security`), l'opération s'effectue en deux étapes immédiates sans modification de code Java :

```
-- Étape A : Renommage atomique instantané au niveau PostgreSQL
ALTER SCHEMA incidentflow RENAME TO core_incidents;
ALTER SCHEMA keycloak RENAME TO iam_security;
```

```
# Étape B : Ajustement déclaratif des variables d'environnement (docker-compose / properties)
spring.jpa.properties.hibernate.default_schema=core_incidents
KC_DB_SCHEMA: iam_security
```